

PH3120

Computational
Physics Laboratory 1

CPL103– Ordinary
Differential Equations

Section 1 & 2

G.W.T. Senesh

2022s19694

1. I. ODEs are equations that includes functions and their derivatives. They describe the relationship between a function and its rate of change.

ODEs general form:

$$F(x, y, y', y'', \dots, y^{(n)}) = 0$$

II. The fundamental difference between an Ordinary Differential Equation (ODE) and a Partial Differential Equation (PDE) lies in the number of independent variables involved in the equation. In ODEs there are one independent variable and PDE has two or more independent variables. Also, ODE use ordinary derivatives and PDE use partial derivative.

III. Key components in ODE are,

- Unknown Function
- Independent Variable
- Coefficients
- Differential Operation
- Order
- Equation Structure

IV. Highest derivative in the equation.

If the equation involves only the first derivative $\frac{dy}{dx}$, it is a first-order ODE.

If it includes the second derivative $\frac{d^2y}{dx^2}$, it is a second-order ODE.

V. $F(x, y, y') = 0$

$$\frac{dy}{dx} + P(x)y = Q(x)$$

VI. $F(x, y, y', y'') = 0$

$$\frac{d^2y}{dx^2} + Q(x)\frac{dy}{dx} + L(x)y = S(x)$$

VII. An Initial Value Problem (IVP) is a type of differential equation along with specified values for the unknown function at a given point in its domain. The goal is to find a function that satisfies the differential equation and meets the initial conditions.

An Initial Value Problem for a first-order differential equation is typically written as:

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0$$

2. I. (a) Euler's Method is derived from the concept of using the tangent line to approximate the solution of an Ordinary Differential Equation (ODE) at discrete points.

$$(b) y(t + dt) = y(t) + y'(t)(dt) + \frac{y''(t)}{2!}(dt)^2 + \frac{y'''(t)}{3!}(dt)^3 + \dots$$

(c) Euler method derivation,

$$y(t + dt) = y(t) + y'(t)(dt) + \frac{y''(t)}{2!}(dt)^2 + \frac{y'''(t)}{3!}(dt)^3 + \dots$$

dt is very small. So, in higher order terms $((dt)^2, (dt)^3, \dots)$ are negligible.

So,

$$y(t + dt) = y(t) + y'(t)(dt)$$

- (d) Local truncation error measures how much the numerical approximation deviates

From the exact solution within a single step.

$$\text{LTE} = \frac{y''(t)}{2!}(dt)^2$$

- (e) Local truncation error measures how much the numerical approximation deviates

From the exact solution within multiple steps.

$$\text{GTE} = O(h)$$

- (f) Euler method is basically designed for first Order ordinary differential equations.

If we want to apply it to 2nd or 3rd order equation, we want to reduce them to a 1st order equation. After converting equation to a first order equation, then use Euler's Method.

- (g) Euler's Method is easy to use and understand. So, it is simple to implement. It is also, foundation for more advanced methods and it can be applied to a wide range of first-order ordinary differential equations, including initial value problems. While its accuracy might be limited; it provides a starting point for problems where an analytical solution is difficult or impossible to find.

(h) To achieve moderate accuracy, Euler's method often requires a very small step size.

So, this method is step sensitivity. Also, for certain types of differential equations,

Euler's method can become numerically unstable.

Error accumulation is another limitation for this method. This can lead to a

Significant drift from the true solution.

II.

```
def euler_method(f, x0, t0, dt, num_step):  
  
    t = [t0]  
    x = [x0]  
  
    t_1 = t0  
    x_1 = x0  
  
    for _ in range(num_step):  
        dx_dt = f(t_1, x_1)  
        x_2 = x_1 + dt*(dx_dt)  
        t_2 = t_1 + dt  
  
        t.append(t_2)  
        x.append(x_2)  
  
        t_1 = t_2  
        x_1 = x_2  
  
    return t,x
```

III. (a)

$$\frac{dx}{dt} = ax - bx^2$$

$$\frac{dx}{dt} = ax \left(1 - \frac{b}{a}x\right)$$

Common form of equation,

$$\frac{dx}{dt} = rx \left(1 - \frac{x}{K_c}\right)$$

so, Let

$$r = a \quad \text{and} \quad K_c = a/b$$

solution,

$$x(t) = \frac{K_c}{1 + \left(\frac{K_c}{x_0} - 1\right)e^{-rt}}$$

$$r = 0.1 \quad \text{and} \quad K_c = a/b = \frac{0.1}{0.001} = 100$$

$$x_0 = 2000$$

$$\begin{aligned} \text{in, } x(4) &= \frac{100}{1 + \left(\frac{100}{2000} - 1\right)e^{-0.4}} \\ &= 275.31 \end{aligned}$$

(b)

Q2(iii)

```
def euler_method(f, x0, t0, dt, num_step):

    t = [t0]
    x = [x0]

    t_1 = t0
    x_1 = x0

    for _ in range(num_step):
        dx_dt = f(t_1, x_1)
        x_2 = x_1 + dt*(dx_dt)
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x

def bac_growth(t, x):
    a = 0.1
    b = 0.001
    return a*x - b*x**2

x0 = 2000
t0 = 0
total_time = 4
num_step = 500

dt = total_time/ num_step
time_values, population_values = euler_method(bac_growth, x0, t0, dt, num_step)

final_population_euler = population_values[-1]
final_time_euler = time_values[-1]

print(f"Analytical solution for x(4) = {275.31:.2f}")
print(f"Euler's Method result at t = {final_time_euler:.2f} hours: {final_population_euler:.2f} bacteria")
```

Analytical solution for x(4) = 275.31
Euler's Method result at t = 4.00 hours: 274.49 bacteria

$$(c) \text{ Percentage error} = \frac{275.31 - 274.49}{275.31} * 100\%$$

0.29%

IV.

$$a) \quad I = C \cdot \frac{dv}{dt}$$

$$I = \frac{V_{in} - V}{R}$$

So,

$$\frac{V_{in} - V}{R} = C \cdot \frac{dv}{dt}$$

$$\frac{V_{in} - V}{RC} = \frac{dv}{dt}$$

$$V_{in} = 5, \quad RC = 2$$

Therefore,

$$\frac{5 - V}{2} = \frac{dv}{dt}$$

Euler's Method formula,

$$V_{n+1} = V_n + h \cdot f(t, V)$$

From ODE,

$$f(t, V) = \frac{5 - V}{2}$$

$$\text{So, } V_{n+1} = V_n + h \cdot \left(\frac{5 - V}{2} \right)$$

(b)

Calculation when $h = 1$;

$$V_0 = 0 \quad t = 1$$

$$\text{so, } V_1 = V_0 + \left(\frac{5 - V_0}{2} \right)$$

$$V_1 = 2.5 \text{ V} //$$

$$t = 2 ,$$

$$V_2 = V_1 + \left(\frac{5 - V_1}{2} \right)$$

$$\begin{aligned} V_2 &= 2.5 + \frac{2.5}{2} \\ &= 3.75 \text{ V} // \end{aligned}$$

$$t = 3$$

$$V_3 = V_2 + \left(\frac{5 - V_2}{2} \right)$$

$$= 3.75 + \frac{1.25}{2}$$

$$= 4.375 \text{ V} //$$

$$t = 4$$

$$V_4 = V_3 + \left(\frac{5 - V_3}{2} \right)$$

$$= 4.375 + \frac{0.625}{2}$$

$$= 4.6875 \text{ V} //$$

Estimate value of V when $t = 4$ is ,
 $4.6875 \text{ V} //$

(c)

```
def euler_method(f, x0, t0, dt, num_step):

    t = [t0]
    x = [x0]

    t_1 = t0
    x_1 = x0

    for _ in range(num_step):
        dx_dt = f(t_1, x_1)
        x_2 = x_1 + dt*(dx_dt)
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x

def voltage(t, x):
    v_in = 5
    rc = 2
    return (v_in-x)/rc

x0 = 0
t0 = 0
total_time = 4
num_step = 500

dt = total_time/ num_step
time_values, v_out = euler_method(voltage, x0, t0, dt, num_step)

final_voltage_euler = v_out[-1]
final_time_euler = time_values[-1]

print(f"Analytical solution for x(4) = {4.6875:.4f}")
print(f"Euler's Method result at t = {final_time_euler:.2f} hours: {final_voltage_euler:.4f} V")

Analytical solution for x(4) = 4.6875
Euler's Method result at t = 4.00 hours: 4.3260 V
```

$$\text{(d) Percentage error} = \frac{4.6875 - 4.3260}{4.6875} * 100\%$$
$$7.72\%$$

V.

a) ←

$$F = m \cdot \frac{d^2x}{dt^2}$$

$$-kx - bv = m \cdot \frac{d^2x}{dt^2}$$

$$-kx - b \cdot \frac{dx}{dt} = m \frac{d^2x}{dt^2}$$

$$m \cdot \frac{d^2x}{dt^2} + b \cdot \frac{dx}{dt} + kx = 0$$

b) Let, $y_1 = x$ and $y_2 = \frac{dx}{dt}$

$$\text{so, } \frac{dy_1}{dt} = y_2$$

$$m \cdot \frac{dy_2}{dt} = -b y_2 - k y_1$$

$$\frac{dy_2}{dt} = -\frac{b}{m} y_2 - \frac{k}{m} y_1$$

in Euler method,

$$v_{n+1} = v_n + h \cdot \left(-\frac{b}{m} y_2 - \frac{k}{m} y_1 \right)$$

$$\text{so, } v_{n+1} = v_n + h \cdot \left(-\frac{b}{m} v - \frac{k}{m} x \right)$$

$$b^2 = 4mk$$

Let, $m = 1 \text{ kg}$ and $k = 1 \text{ Nm}^{-1}$

$$\text{so, } b^2 = 4$$

$$b = 2$$

with those values,

$$\frac{b}{m} = \frac{2}{1} = 2$$

$$\frac{k}{m} = \frac{1}{1} = 1$$

$$\text{So, } x_{n+1} = x_n + h \cdot v_n$$

$$v_{n+1} = v_n + h \cdot (-2v - 1x)$$

$$\text{in } t = 0, v = 0$$

$$\text{so, } v_1 = 0 + h \cdot (-2 \times 0 - 1 \times 10)$$

$$\text{for } h = 1$$

$$v_1 = -10 \text{ cm s}^{-1} //$$

$$t = 2; x_2 = x_1 + 1(v_1)$$

$$v_2 = v_1 + 1(-2 \times v_1 - 1 \times 0)$$

$$= -10 + (-2) \times (-10)$$

$$= -10 + 20$$

$$= 10 \text{ cm s}^{-1} //$$

$$\text{For, } t = 0.1 \text{ s}$$

$$x_1 = x_0 + 0.1 v_0$$

$$x_1 = 10 \text{ cm}$$

$$v_1 = v_0 + h \cdot (-2v_0 - 1x_0)$$

$$= 0 + 0.1(-2 \times 0 - 10)$$

$$v_1 = -1 \text{ cm s}^{-1} //$$

For, $t = 0.2 \text{ s}$

$$\begin{aligned}x_2 &= x_1 + 0.1 v_1 \\&= 10 + 0.1 \times (-1) \\&= 10 - 0.1 \\&= 9.9 \text{ cm}\end{aligned}$$

$$\begin{aligned}v_2 &= v_1 + h \cdot (-2v_1 - 1 \times x_1) \\&= -1 + 0.1 (-2 - (+10)) \\&= -1 + 0.1 (-8) \\&= -1.8 \text{ cm s}^{-1}\end{aligned}$$

To plot $x(t)$

$$m \cdot \frac{d^2x}{dt^2} + b \cdot \frac{dx}{dt} + kx = 0$$

solution assume, $x(t) = e^{rt}$

$$m \cdot r^2 + br + k = 0$$

$$r = \frac{-b \pm \sqrt{b^2 - 4mk}}{2m}$$

$$b^2 = 4mk$$

so, $r = \frac{-b}{2m}$, so this case has repeated roots.

Therefore,

$$x(t) = (c_1 + c_2 t) e^{rt}$$

$$r = -\frac{b}{2m}$$

$$x(t) = (c_1 + c_2 t) e^{-\frac{b}{2m} t}$$

in initial point,

$$t = 0, \quad x = 10$$

$$10 = (c_1 + 0) e^0$$

$$c_1 = 10$$

Find,

$$\frac{d x(t)}{d t} = (c_1 + c_2 t) \left(-\frac{b}{2m} e^{-\frac{b}{2m} t} \right) + e^{-\frac{b}{2m} t} \times c_2$$

$$\text{at } t = 0, \quad \frac{d x(t)}{d t} = 0$$

so,

$$0 = (10 +) \left(-\frac{b}{2m} \right) e^0 + e^{-\frac{b}{2m} \times 0} \times c_2$$

$$c_2 = \frac{10 b}{2m}$$

$$\text{so, } x(t) = \left(10 + 10 \cdot \frac{b}{2m} t \right) e^{-\frac{b}{2m} t}$$

$$x(t) = \left(10 + \frac{5b}{m} t \right) e^{-\frac{b}{2m} t}$$

```

import numpy as np
import matplotlib.pyplot as plt

m = 1.0
b = 2.0

def x_analytical(t, m, b):
    return (10 + (5 * b / m) * t) * np.exp(-(b / (2 * m)) * t)

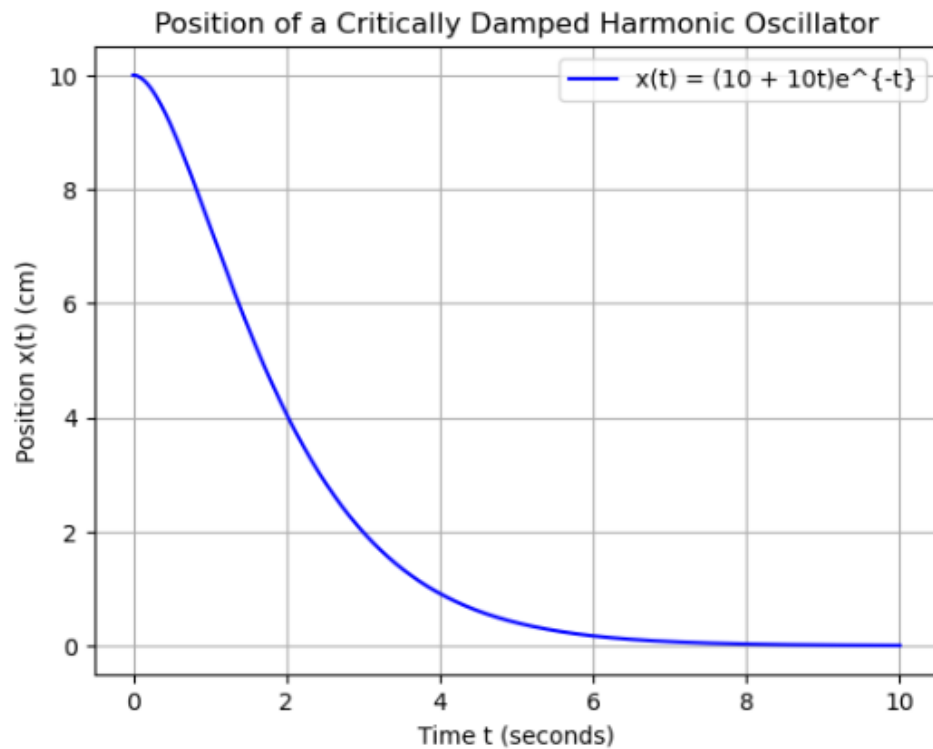
t_start = 0
t_end = 10
t_values = np.linspace(t_start, t_end, 500)

x_values = x_analytical(t_values, m, b)

plt.plot(t_values, x_values, label=r' $x(t) = (10 + 10t)e^{-t}$ ', color='blue')

plt.title('Position of a Critically Damped Harmonic Oscillator')
plt.xlabel('Time t (seconds)')
plt.ylabel('Position x(t) (cm)')
plt.grid()
plt.legend()
plt.show()

```



(c)

```
def euler_method(f, x0_vector, t0, dt, num_step):
```

```
    t = [t0]
    x_states = [np.array(x0_vector)]
```

```
    t_1 = t0
    x_1 = np.array(x0_vector)
```

```
    for _ in range(num_step):
        dx_dt = f(t_1, x_1)
        x_2 = x_1 + dt*dx_dt
        t_2 = t_1 + dt
```

```
        t.append(t_2)
        x_states.append(x_2)
```

```
    t_1 = t_2
    x_1 = x_2
```

```
    return t, x_states
```

```
def damped_oscillator(t, state):
```

```
    x = state[0]
    v = state[1]
    b = 2.0
    m = 1.0
    k = 1.0
    dxdt = v
    dvdt = (-b/m)*v - (k/m)*x
    return np.array([dxdt, dvdt])
```

```
x0_position = 10.0
```

```
x0_velocity = 0.0
```

```
state_vector = np.array([x0_position, x0_velocity])
```

```
t0 = 0.0
```

```
total_time = 0.2
```

```
num_step = 500
```

```
dt = total_time/ num_step
```

```
time_values, state_vectors = euler_method(damped_oscillator, state_vector, t0, dt, num_step)
```

```
position_values = [state[0] for state in state_vectors]
```

```
velocity_values = [state[1] for state in state_vectors]
```

```
print(f"\nPosition at t={time_values[-1]:.2f}s (last step): {position_values[-1]:.4f} cm")
```

```
print(f"Velocity at t={time_values[-1]:.2f}s (last step): {velocity_values[-1]:.4f} cm/s")
```

Position at t=0.20s (last step): 9.8250 cm

Velocity at t=0.20s (last step): -1.6381 cm/s

3. I.

03) 1. a) Together with Taylor series,

$$y_{n+1} = y_n + \omega_1 k_1 + \omega_2 k_2$$

$$y_{n+1} = y_n + h(\omega_1 + \omega_2)f + h^2\omega_3(a f_x + b f_y) + \dots$$

$$y_{n+1} = y_n + \omega_1 h f(x_n, y_n) + \omega_2 h f(x_n + ah, y_n + bh)$$

↑
This is the second

Then, order R-K equation.
 $\omega_1 + \omega_2 = 1$

when, $\omega_2 = 0 \rightarrow$ Euler's method.

Therefore Runge - kutta is a generalization of the Euler's method.

so, R-K method for 2nd order

$$y_{n+1} = y_n + \omega_1 h f(x_n, y_n) + \omega_2 h f(x_n + ah, y_n + bh)$$

with $\omega_1 + \omega_2 = 1$.

where,

$$k_1 = h \cdot f(x_n, y_n)$$

$$\begin{aligned} \text{b) Truncation Error} &= y(t_{n+1}) - y_{n+1} \\ &= O(h^3) \end{aligned}$$

(c) This method has Higher accuracy than basic Euler's method, the second-order Runge-kutta method provides a more accurate approximation of the true solution.

Also, this method has Improved stability. So, this allows for the use of larger step sizes.

RK method gives the better approximation of the slope.

(d) The Local Truncation Error is the error made by a numerical method in a single step, assuming that the solution at the beginning of the step.

So, the LTE is the difference,

$$\begin{aligned}LTE &= y(t_{n+1}) - y_{n+1} \\LTE &= O(h^3)\end{aligned}$$

The Global Truncation Error is the total accumulated error over the entire integration Interval, from the starting point up to given time.

$$\begin{aligned}GTE &= y(t_N) - y_N \\GTE &= O(h^2)\end{aligned}$$

(e) When the step size is decreasing, Local Truncation Error and Global Truncation Error of the RK method also decreasing. So, when the step size is small the accuracy of the method is Higher.

II.

```
import numpy as np
def runge_kutta_second_order(f, x0, t0, dt, num_step):

    t = [t0]
    x = [np.array(x0)]

    t_1 = t0
    x_1 = np.array(x0)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + dt, x_1 + k_1)
        t_2 = t_1 + dt

        x_2 = x_1 + 0.5 * (k_1 + k_2)
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x
```

III. 2(III)

```
import numpy as np
def runge_kutta_second_order(f, x0, t0, dt, num_step):

    t = [t0]
    x = [np.array(x0)]

    t_1 = t0
    x_1 = np.array(x0)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + dt, x_1 + k_1)
        t_2 = t_1 + dt

        x_2 = x_1 + 0.5 * (k_1 + k_2)
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x

def bac_growth(t, x):
    a = 0.1
    b = 0.001
    return a*x - b*x**2

x0 = 2000
t0 = 0
total_time = 4
num_step = 500

dt = total_time/ num_step
time_values, population_values = runge_kutta_second_order(bac_growth, x0, t0, dt, num_step)

final_population_RK = population_values[-1]
final_time_RK = time_values[-1]

print(f"Analytical solution for x(4) = {275.31:.2f}")
print(f"RK method second order at t = {final_time_euler:.2f} hours: {final_population_euler:.2f} bacteria")

Analytical solution for x(4) = 275.31
RK method second order at t = 4.00 hours: 275.34 bacteria
```

2(IV)

```
import numpy as np
def runge_kutta_second_order(f, x0, t0, dt, num_step):

    t = [t0]
    x = [np.array(x0)]

    t_1 = t0
    x_1 = np.array(x0)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + dt, x_1 + k_1)
        t_2 = t_1 + dt

        x_2 = x_1 + 0.5 * (k_1 + k_2)
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x
def voltage(t, x):
    v_in = 5
    rc = 2
    return (v_in-x)/rc

x0 = 0
t0 = 0
total_time = 4
num_step = 500

dt = total_time/ num_step
time_values, v_out = runge_kutta_second_order(voltage, x0, t0, dt, num_step)

final_voltage_RK = v_out[-1]
final_time_RK = time_values[-1]

print(f"Analytical solution for x(4) = {4.6875:.4f}")
print(f"RK method second order result at t = {final_time_RK:.2f} hours: {final_voltage_RK:.4f} V")
```

Analytical solution for $x(4) = 4.6875$
RK method second order result at $t = 4.00$ hours: 4.3233 V

2(V)

```
import numpy as np
def runge_kutta_second_order(f, x0_vector, t0, dt, num_step):
```

```
    t = [t0]
    x_state = [np.array(x0_vector)]

    t_1 = t0
    x_1 = np.array(x0_vector)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + dt, x_1 + k_1)
        t_2 = t_1 + dt

        x_2 = x_1 + 0.5 * (k_1 + k_2)
        t_2 = t_1 + dt

        t.append(t_2)
        x_state.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t, x_state
```

```
def damped_oscillator(t, state):
    x = state[0]
    v = state[1]
    b = 2.0
    m = 1.0
    k = 1.0
    dxdt = v
    dvdt = (-b/m)*v - (k/m)*x
    return np.array([dxdt, dvdt])
```

```
x0_position = 10.0
x0_velocity = 0.0
state_vector = np.array([x0_position, x0_velocity])
t0 = 0.0
total_time = 0.2
num_step = 500

dt = total_time/ num_step
time_values, state_vectors = runge_kutta_second_order(damped_oscillator, state_vector, t0, dt, num_step)

position_values = [state[0] for state in state_vectors]
velocity_values = [state[1] for state in state_vectors]

print(f"\nPosition at t={time_values[-1]:.2f}s (last step): {position_values[-1]:.4f} cm")
print(f"Velocity at t={time_values[-1]:.2f}s (last step): {velocity_values[-1]:.4f} cm/s")
```

Position at t=0.20s (last step): 9.8248 cm
Velocity at t=0.20s (last step): -1.6375 cm/s

4. I. (a)

a)

$$y(t+h) = y(t) + hy'(t) + \frac{h^2}{2!} y''(t) + \frac{h^3}{3!} y'''(t) + \frac{h^4}{4!} y^{(4)}(t) + o(h^5)$$

$$y'(t) = f$$

$$y''(t) = f_t + f f_y$$

$$y'''(t) = f_{tt} + 2f f_{ty} + f_y(t + f f_y)$$

$$y^{(4)}(t) = \text{higher terms.}$$

For RK 4 method approximates the integral of (t, y) -

$$k_1 = hf(t_n, y_n)$$

$$k_2 = hf(t_n + a_2 h, y_n + b_2 k_1)$$

$$k_3 = hf(t_n + a_3 h, y_n + b_3 k_1 + c_3 k_2)$$

$$k_4 = hf(t_n + a_4 h, y_n + b_4 k_1 + c_4 k_2 + d_4 k_3)$$

$$y_{n+1} = y_n + w_1 k_1 + w_2 k_2 + w_3 k_3 + w_4 k_4$$

Final formula,

$$y_{n+1} = y_n + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

(b) Local Truncation Error (LTE): This error for 4th order Runge-kutta method is order $O(h^5)$.

Global Truncation Error (GTE): This error for 4th order Runge-kutta method is order $O(h^4)$.

(c) Euler's method is a first-order method. So, Runge-kutta fourth order method has much higher accuracy. Therefore, Runge kutta 4th order method used to solve the larger step size ODE's.

Also, RK4 method generally possess better stability compare to the Euler's method.

(d) The fourth-order Runge-Kutta method offers significant advantages over the second-order Runge-Kutta method, primarily in terms of accuracy and efficiency for achieving high accuracy.

IV.

```
import numpy as np
def runge_kutta_fourth_order(f, x0, t0, dt, num_step):

    t = [t0]
    x = [np.array(x0_vector)]

    t_1 = t0
    x_1 = np.array(x0)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_1)
        k_3 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_2)
        k_4 = dt*f(t_1 + dt, x_1 + k_3)

        x_2 = x_1 + (k_1 + 2*k_2 + 2*k_3 + k_4)/6.0
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t, x_state
```

V.
(2(III)).

```
import numpy as np
def runge_kutta_fourth_order(f, x0, t0, dt, num_step):

    t = [t0]
    x = [np.array(x0)]

    t_1 = t0
    x_1 = np.array(x0)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_1)
        k_3 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_2)
        k_4 = dt*f(t_1 + dt, x_1 + k_3)

        x_2 = x_1 + (k_1 + 2*k_2 + 2*k_3 + k_4)/6.0
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x

def bac_growth(t, x):
    a = 0.1
    b = 0.001
    return a*x - b*x**2

x0 = 2000
t0 = 0
total_time = 4
num_step = 500

dt = total_time/ num_step
time_values, population_values = runge_kutta_fourth_order(bac_growth, x0, t0, dt, num_step)

final_population_RK = population_values[-1]
final_time_RK = time_values[-1]

print(f"Analytical solution for x(4) = {275.31:.2f}")
print(f"RK method fourth order at t = {final_time_RK:.2f} hours: {final_population_RK:.2f} bacteria")

Analytical solution for x(4) = 275.31
RK method fourth order at t = 4.00 hours: 275.33 bacteria
```

2(IV)

```
import numpy as np
def runge_kutta_fourth_order(f, x0, t0, dt, num_step):

    t = [t0]
    x = [np.array(x0)]

    t_1 = t0
    x_1 = np.array(x0)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_1)
        k_3 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_2)
        k_4 = dt*f(t_1 + dt, x_1 + k_3)

        x_2 = x_1 + (k_1 + 2*k_2 + 2*k_3 + k_4)/6.0
        t_2 = t_1 + dt

        t.append(t_2)
        x.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x

def voltage(t, x):
    v_in = 5
    rc = 2
    return (v_in-x)/rc

x0 = 0
t0 = 0
total_time = 4
num_step = 500

dt = total_time/ num_step
time_values, v_out = runge_kutta_fourth_order(voltage, x0, t0, dt, num_step)

final_voltage_RK = v_out[-1]
final_time_RK = time_values[-1]

print(f"Analytical solution for x(4) = {4.6875:.4f}")
print(f"RK method fourth order result at t = {final_time_RK:.2f} hours: {final_voltage_RK:.4f} V")
```

Analytical solution for $x(4) = 4.6875$
RK method fourth order result at $t = 4.00$ hours: 4.3233 V

(2(V))

```
import numpy as np
def runge_kutta_fourth_order(f, x0_vector, t0, dt, num_step):
```

```
    t = [t0]
    x_state = [np.array(x0_vector)]

    t_1 = t0
    x_1 = np.array(x0_vector)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_1)
        k_3 = dt*f(t_1 + 0.5 * dt, x_1 + 0.5*k_2)
        k_4 = dt*f(t_1 + dt, x_1 + k_3)

        x_2 = x_1 + (k_1 + 2*k_2 + 2*k_3 + k_4)/6.0
        t_2 = t_1 + dt

        t.append(t_2)
        x_state.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t, x_state
```

```
def damped_oscillator(t, state):
```

```
    x = state[0]
    v = state[1]
    b = 2.0
    m = 1.0
    k = 1.0
    dxdt = v
    dvdt = (-b/m)*v - (k/m)*x
    return np.array([dxdt, dvdt])
```

```
|
x0_position = 10.0
```

```
x0_velocity = 0.0
```

```
state_vector = np.array([x0_position, x0_velocity])
```

```
t0 = 0.0
```

```
total_time = 0.2
```

```
num_step = 500
```

```
dt = total_time / num_step
```

```
time_values, state_vectors = runge_kutta_fourth_order(damped_oscillator, state_vector, t0, dt, num_step)
```

```
position_values = [state[0] for state in state_vectors]
```

```
velocity_values = [state[1] for state in state_vectors]
```

```
print(f"\nPosition at t={time_values[-1]:.2f}s (last step): {position_values[-1]:.4f} cm")
```

```
print(f"Velocity at t={time_values[-1]:.2f}s (last step): {velocity_values[-1]:.4f} cm/s")
```

Position at t=0.20s (last step): 9.8248 cm

Velocity at t=0.20s (last step): -1.6375 cm/s

5. I. (2(III))

```
import numpy as np
from scipy.integrate import solve_ivp

a = 0.1
b = 0.001
x0 = 2000

def analytical_solution_bacteria(t, x0, a, b):

    num = x0 * a * np.exp(a * t)
    d_num = (a - b * x0) + b * x0 * np.exp(a * t)
    return num / d_num

def bac_growth_ode(t, x):
    return a * x - b * x**2

x0_bacteria = 2000
t0_bacteria = 0
total_time_bacteria = 4

t_span = (t0_bacteria, total_time_bacteria)
t_eval = [total_time_bacteria]

solution_ivp = solve_ivp(bac_growth_ode, t_span, [x0_bacteria], t_eval=t_eval, method='RK45')
numerical_bacteria_at_4h = solution_ivp.y[0][-1]
analytical_bacteria_at_4h = analytical_solution_bacteria(total_time_bacteria, x0, a, b)

if analytical_bacteria_at_4h != 0:
    percentage_error = abs((numerical_bacteria_at_4h - analytical_bacteria_at_4h) / analytical_bacteria_at_4h) * 100
else:
    percentage_error = float('inf')

print(f"\nAnalytical solution for bacteria after {total_time_bacteria} hours: {analytical_bacteria_at_4h:.2f}")
print(f"Numerical solution (SciPy solve_ivp) for bacteria after {total_time_bacteria} hours: {numerical_bacteria_at_4h:.2f}")
print(f"Percentage Error: {percentage_error:.4f}%")
```

Analytical solution for bacteria after 4 hours: 275.33
Numerical solution (SciPy solve_ivp) for bacteria after 4 hours: 275.82
Percentage Error: 0.1784%